Structured Schemas for LLM-Modeler Collaboration in QSP Model Calibration

Joel Eliason[1] and Aleksander S. Popel[1]

[1]Department of Biomedical Engineering, Johns Hopkins University, Baltimore, MD, USA

Abstract

Quantitative systems pharmacology (QSP) models require parameter values derived from experimental literature, yet current approaches face a trade-off between throughput and rigor. Manual curation remains the gold standard but often produces inconsistent documentation, with uncertainty quantification and provenance not always tracked systematically. Large language models (LLMs) can accelerate literature search and extraction, but exhibit failure modes including hallucinated values and citation fabrication that are unacceptable for quantitative modeling. We present MAPLE (Model-Aware Parameter Literature Extraction), a framework that uses structured validation schemas as a collaboration interface between LLMs and modelers. Two complementary schemas capture calibration data at different scales: one for isolated experiments that constrain individual parameters through simplified forward models (mathematical representations linking measurements to parameters), and one for clinical and in vivo endpoints that constrain the full model through species-level observables. Both schemas separate data extraction from modeling decisions, capturing literature values with full provenance while constraining output to a machine-verifiable form. Targeted validators catch characteristic LLM errors: value-in-snippet matching detects hallucinated values, DOI resolution flags fabricated citations, and code execution catches malformed forward models. We evaluate MAPLE on 87 calibration targets for a pancreatic ductal adenocarcinoma (PDAC) QSP model spanning tumor, stromal, and immune cell dynamics, using two collaboration modes: batch LLM extraction followed by interactive modeler curation, and interactive extraction where modeler and LLM collaborate in real time. Both modes required substantial modeler input. No batch extraction was used without modification: the modeler changed the forward model type in 65% of SubmodelTargets, adjusted prior distribution parameters in 46%, and revised source relevance

assessments in all files. Interactively extracted targets required comparable modeler effort but embedded it in the extraction process, producing near-final output with minimal subsequent revision. The schemas structure the collaboration in both modes, ensuring completeness and enabling reproducible, provenance-rich calibration regardless of workflow.

1 Introduction

Quantitative systems pharmacology models translate mechanistic understanding of disease biology into predictions of drug response [1]. QSP models are used across therapeutic areas, from immuno-oncology [2, 3] to autoimmune disease and metabolic disorders, with shared challenges in parameter calibration. Capturing mechanistic detail across biological scales can require models with many components: some QSP models comprise tens to hundreds of ordinary differential equations representing cell populations, signaling molecules, and drug pharmacokinetics. Clinical endpoints are too sparse and aggregated to identify the individual parameters governing these dynamics; instead, values for proliferation rates, death rates, and binding affinities must come from targeted experimental studies reported in the literature. Because models are used for prediction, calibration must quantify uncertainty: virtual population generation and virtual clinical trials depend on parameter distributions, not point estimates [4, 5]. As the field increasingly integrates QSP with machine learning and artificial intelligence [6, 7, 8, 9, 10], the bottleneck of manual parameter extraction becomes more acute: automated model development requires automated access to calibration data.

Beyond identifying relevant literature lies the challenge of extracting data systematically with full provenance. Each experiment reports results in different formats, with different units, time points, and uncertainty quantification. Experimental conditions vary across studies and are often inconsistently documented. Converting a reported measurement into a calibration constraint requires judgment about applicability, uncertainty, and potential biases. The reasoning behind each extraction (why this value, from this paper, interpreted this way) must be documented for reproducibility and review.

The biological literature contains a wealth of experiments that isolate specific mechanisms from the complexity of in vivo systems. In vitro proliferation assays measure cell division rates without

immune infiltration or nutrient gradients. Ex vivo cytotoxicity experiments quantify killing efficiency without the confounding effects of tumor heterogeneity or drug distribution. These isolated system experiments are valuable precisely because they remove confounding factors: e.g., when a proliferation assay reports a doubling time, that measurement directly constrains the proliferation rate parameter. We use the term *calibration target* to refer to a structured representation of such experimental data, capturing what was measured, under what conditions, with what uncertainty, and how that measurement relates to model parameters.

Manual curation by modelers remains the standard approach to extracting calibration targets from literature. An experienced modeler must understand what parameters the model needs, search for experiments that constrain those parameters, identify relevant passages, assess methodological quality, judge applicability, and document assumptions. The result, when done well, is a well-reasoned parameter estimate with clear provenance. However, manual curation has significant limitations. The process is labor-intensive. Documentation quality varies across individuals and projects, with uncertainty quantification and provenance tracking handled inconsistently if at all. When personnel change, institutional knowledge disappears, and the reasoning behind a parameter choice may be lost even if the value persists in the model. A further challenge arises when combining constraints from multiple experiments: without a structured representation, joint inference requires ad-hoc scripting for each new combination of experiments, with no standard path from extracted data to inference code.

Automated extraction has been a goal of biomedical text mining for decades [11], but traditional natural language processing (NLP) pipelines struggle with the distributed, context-dependent information that characterizes experimental reports. Recent work has developed specialized NLP models for biomedical named entity recognition and relation extraction, with transformer-based approaches like BioBERT achieving high accuracy on domain-specific extraction tasks [12, 13]. Deep learning approaches have also been applied to data element extraction for systematic literature reviews [14]. For pharmacokinetic parameters specifically, active learning approaches have produced annotated corpora and fine-tuned language models for NER of PK terms [15]. However, these approaches require task-specific training data and do not generalize easily to the diverse parameter types encountered in QSP modeling.

Large language models offer a more flexible path forward. LLMs with web search capabilities can

identify relevant literature when given appropriate context about what parameters need calibration and how they are used mechanistically in the model. Once papers are identified, LLMs can read scientific text, follow structured templates, and extract information at scale. Recent work has demonstrated LLM-based extraction for biomedical knowledge graphs [16], systematic literature reviews [17], clinical endpoints [18], and QSP model development [19]. Dedicated software packages now exist for LLM-based biomedical information extraction [20], and interactive systems can extract structured data from scientific literature into structured formats [21].

Nevertheless, LLMs have well-documented failure modes that are particularly problematic for quantitative scientific work. Hallucination (the generation of plausible but incorrect information) is endemic to current models [22, 23]. LLMs may fabricate numeric values, invent citations to papers that do not exist, or generate internally inconsistent outputs. These failures are especially dangerous because they often appear fluent and confident [24]. Mitigation strategies include retrieval-augmented generation (RAG) and reasoning enhancement [25], but for parameter extraction, where the extracted values directly influence model predictions, naive LLM extraction without validation is not acceptable.

Many of the failure modes of LLM extraction, however, are amenable to systematic detection. Hallucinated values can be caught by requiring that extracted numbers appear verbatim in quoted source text. Fabricated citations can be identified through DOI resolution. Internal inconsistencies can be flagged through schema validation. This suggests a workflow where LLMs handle tasks they do well (reading scientific text and following structured templates) while automated validators catch their characteristic errors before human review. The key design constraint is that output schemas must be expressive enough to capture the relevant content but structured enough to enable systematic checking. Pydantic [26], a Python data validation library, provides this capability: when LLM output fails validation, the error message can be automatically fed back to the LLM for retry, creating a self-correcting loop that reduces the burden on human reviewers.

We present MAPLE (Model-Aware Parameter Literature Extraction), a framework for LLM-assisted calibration of QSP models with four components: (1) model-aware literature search, where prompts inject mechanistic context (parameter descriptions, reactions, rate laws) to guide LLM web search toward experiments that can constrain specific parameters; (2) validated YAML schemas for two calibration regimes (isolated experiments constraining individual parameters, and clinical/in

vivo endpoints constraining the full model), both separating data extraction from model specification while preserving traceability from posterior to source text; (3) a validation framework targeting specific failure modes, including value-in-snippet matching for hallucination detection and DOI resolution for citation verification; and (4) code generation for downstream Bayesian inference. We evaluate MAPLE on calibration targets for a PDAC QSP model, covering tumor, stromal, and immune cell dynamics, and report quantitative metrics on validation performance and extraction reliability.

2 Methods

2.1 Model-Aware Literature Search

The workflow begins with a parameter-driven search: the user specifies which model parameters need calibration, and the system builds a prompt that guides an LLM with web search capability to find relevant experimental literature. The framework is not specific to any particular LLM: it supports multiple providers (OpenAI, Anthropic, Google) through a common interface and allows configuration of model version and reasoning effort (extended thinking), enabling users to trade off cost, latency, and extraction quality. In this study, we used GPT-5.1 for batch pipeline extraction and Claude Opus 4.6 for interactive extraction. A preview mode generates prompts without making API calls, allowing users to inspect and modify the prompt before committing to extraction.

The prompt injects mechanistic context extracted from the model structure. For each target parameter, the system automatically retrieves: the parameter's name, units, and biological description; the reactions where the parameter appears with their rate laws; the species involved in those reactions with descriptions; other parameters appearing in the same reactions; and the broader reaction network showing other reactions that involve the same species. For example, when searching for data to calibrate a proliferation rate `k_apsc_prolif`, the prompt includes the reaction `aPSC -> 2*aPSC` with rate law `k_apsc_prolif * aPSC`, species descriptions explaining that aPSC represents activated pancreatic stellate cells, and related parameters like death rates that appear in the same reaction network.

This context guides the LLM's search toward experiments that actually constrain the target parameters, rather than experiments that mention similar concepts but measure different quanti-

ties. The broader reaction network helps the LLM identify whether additional parameters should be jointly calibrated. Extracting multiple parameters from the same study is intentional and preferred when the data supports it, as shared experimental conditions reduce inter-study variability. For example, if a proliferation experiment also reports apoptosis data, the LLM can include the corresponding death rate parameter.

To encourage literature diversity, source exclusion prevents the LLM from reusing papers across extractions. The prompt includes a list of primary sources already used for other calibration targets, and a strict exclusion mode extends this to sources used by related parameters in the same reaction network. This prevents the LLM from repeatedly extracting from the same well-known papers and promotes broader coverage of the experimental literature.

Beyond mechanistic context, the prompt includes the complete schema structure with field descriptions, enabling the LLM to generate valid output without trial and error. The prompt also provides explicit guidance on common pitfalls: distinguishing SEM from SD (with conversion formulas), avoiding hardcoded numeric values in code blocks, and properly documenting source relevance for cross-species or cross-indication extractions. By showing validation requirements upfront (such as the prohibition on non-peer-reviewed sources and required uncertainty thresholds for proxy data), the prompt reduces the retry loop by preventing predictable errors.

Once the LLM identifies a relevant paper, it extracts data into a structured schema that captures both the raw literature values and the modeling decisions needed for inference.

2.2 Schema Design

The framework defines two complementary schemas for two calibration regimes. The *SubmodelTarget* schema captures isolated experiments (in vitro assays, single-mechanism studies) that constrain individual model parameters through simplified forward models. The *CalibrationTarget* schema captures clinical and in vivo endpoints that constrain the full model through observables computed over the complete species state. Both schemas share a common design philosophy but differ in how they connect literature data to model parameters.

2.2.1 Shared Design Principles

Both schemas enforce a data-first architecture: extraction of what the paper reports is separated from specification of how to use it for calibration. Every extracted numeric value must appear verbatim in a quoted text snippet from the source, creating an auditable trail from parameter value to paper and enabling automated detection of hallucinated values. Both schemas require structured source documentation (DOI, title, authors) and source relevance assessment (indication match, species translation, TME compatibility), making translation assumptions explicit and quantifying expected uncertainty.

Both schemas capture experimental context metadata (species, system type, disease stage, treatment status) and structured study interpretation (scientific rationale, key assumptions, known limitations). These fields create a record of modeler judgment that complements the quantitative extraction and supports automated filtering of targets by context.

Both schemas are implemented as typed models with fields and constraints. When YAML is parsed, the framework validates all constraints automatically. When extraction fails validation, structured error messages provide actionable feedback for the LLM retry loop.

2.2.2 SubmodelTarget: Parameter Priors from Isolated Experiments

The SubmodelTarget schema structures calibration targets into two layers: an *inputs* layer that captures data exactly as reported in the literature (value, units, uncertainty, sample size, type classification, source reference, and quoted snippet), and a *calibration* layer that specifies how that data constrains model parameters. Supplementary Section 6 provides detailed field descriptions with representative YAML examples for each layer.

The calibration layer specifies how extracted data constrains model parameters through a forward model that captures only the dynamics relevant to the experiment. For time-course data, this is typically a simplified ODE: a proliferation assay can be modeled as exponential growth ($dy/dt = k \cdot y$) where k is the proliferation rate. For steady-state or derived measurements, an algebraic model may suffice: if a paper reports half-life $t_{1/2}$, the forward model is simply $t_{1/2} = \ln(2)/k$, requiring no ODE integration. In both cases, the submodel derives from the full QSP model by isolating relevant species and treating confounding factors as absent.

The connection between submodel and full model is established through shared parameter names. When a proliferation rate in a submodel is named `k_apsc_prolif`, it refers to the same parameter in the full model. This naming convention allows posteriors from submodel inference to serve directly as priors for full-model calibration without post-hoc mapping.

Each parameter includes a prior distribution (lognormal, normal, uniform, or half-normal) with documented rationale connecting literature values to distribution parameters and explaining the assumed uncertainty range. An error model specifies the likelihood function connecting predictions to data: observation code derives the observed value and uncertainty from extracted inputs, and a likelihood distribution family completes the statistical specification.

Forward model type system. The schema provides 15 built-in forward model types organized in four categories (Supplementary Table 1). Six *ODE templates* cover common experimental dynamics: exponential growth, first-order decay, two-state transitions, logistic growth, saturation, and Michaelis-Menten kinetics. Five *structured steady-state* types encode specific biophysical relationships for equilibrium measurements: cell density to trafficking rate, population fraction to recruitment rate, soluble factor concentration to secretion rate, population ratio to relative rates, and proliferation marker fraction to growth rate. A *batch accumulation* type models in vitro secretion assays. Three *generic* types (algebraic, direct fit, and custom) serve as fallbacks when built-in templates are insufficient.

Each structured model declares its fields as typed roles rather than raw values. A field can reference a calibration parameter (to be estimated), an extracted input (from the inputs layer), a curated reference value from a shared database, or a numeric literal. This role-based design allows the same model template to serve different estimation scenarios depending on which field is treated as the unknown. A unit conversion factor field corrects for mismatched units between parameters, for instance when a target rate is measured per minute but the model's loss rate is defined per day.

A complete SubmodelTarget example is provided in Supplementary Section 1.

2.2.3 CalibrationTarget: Clinical and In Vivo Endpoints

While SubmodelTarget isolates individual mechanisms, many calibration constraints come from clinical or in vivo studies where the measurement reflects the integrated behavior of the full model.

Intratumoral CD8+ T cell density measured by immunohistochemistry, for example, reflects the net effect of recruitment, proliferation, exhaustion, and death, all of which are governed by distinct model parameters. The CalibrationTarget schema represents these full-model endpoints.

The *observable* specifies how to compute the experimental measurement from the full model species. An observable is a Python function `compute_observable(time, species_dict, constants, ureg)` that receives the solved model state and returns a quantity array with units. For example, a CD8+ density observable sums effector and exhausted CD8+ T cell counts, then divides by tumor area estimated from cancer cell count and reference database constants (cell cross-section, cellularity fraction). Named constants carry provenance: each must trace to either a curated reference database or a specific literature source with documented biological basis.

The *empirical data* block derives summary statistics (median and 95% credible interval) from literature inputs via Monte Carlo simulation. A `distribution_code` function receives extracted inputs (means, SEMs, sample sizes) and generates samples from the implied population distribution. This approach handles common complications naturally: combining subgroups as mixtures, converting between SEM and SD, and propagating non-Gaussian uncertainty. The schema supports both scalar endpoints and vector-valued data (time courses, dose-response curves) through optional index fields.

For intervention studies, a *scenario* block captures the treatment context: agents, doses, routes, schedules, treatment line, and prior therapy history. Population-level endpoints (response rates, survival) are handled through an aggregation specification that defines how individual model trajectories map to cohort-level statistics.

A complete CalibrationTarget example is provided in Supplementary Section 7; schema definitions for all CalibrationTarget components are in Supplementary Section 5.

2.2.4 Source Documentation

Each target includes structured metadata about its data sources. The `primary_data_source` block captures the main reference: DOI, title, authors, year, and a short `source_tag` used for cross-referencing elsewhere in the schema. Secondary sources that provide supporting context (e.g., reviews explaining biological mechanisms, or related studies used for uncertainty estimation) are listed in `secondary_data_sources`, each with a `contribution` field explaining how that source

informs the extraction.

2.2.5 Source Relevance Assessment

Experimental data rarely matches the target model perfectly. The `source_relevance` block documents the translation from source context to model context, making assumptions explicit and quantifying expected uncertainty.

The `indication_match` field classifies how well the source disease context matches the model: `exact` (same indication), `related` (same organ or disease class), `proxy` (different tissue used as mechanistic proxy), or `unrelated`. Cross-indication extractions require a justification explaining why the data is still informative.

Species translation is captured by `species_source` and `species_target` fields. When these differ (e.g., mouse data informing a human model), the extraction must document the assumed translation.

The `source_quality` field categorizes the evidence type: `primary_human_clinical`, `primary_human_in_vitro`, `primary_animal_in_vivo`, `primary_animal_in_vitro`, `review_article`, or `textbook`. By default, non-peer-reviewed sources trigger a validation warning; in practice, preprints (e.g., from bioRxiv) that have been publicly available for several months can provide valuable data, and the validator can be configured to accept them with documented justification.

For immune and stromal parameters, `tme_compatibility` assesses whether the tumor microenvironment in the source matches the target model (`high`, `moderate`, or `low`). A T cell trafficking rate measured in melanoma (a T cell-permissive tumor) may not translate directly to PDAC (an immunosuppressive tumor); low compatibility requires explicit documentation.

Finally, `estimated_translation_uncertainty_fold` quantifies the expected uncertainty from all translation factors combined. Cross-species extraction from a proxy indication with low TME compatibility might warrant 10-fold or greater uncertainty, which propagates to prior width during inference.

2.2.6 Schema Implementation

Both schemas are implemented using Pydantic [26], a Python library for declarative data validation. Each schema component is defined as a Pydantic model with typed fields and constraints. When

YAML is parsed, Pydantic validates all constraints automatically: required fields must be present, values must match declared types, enumerations must use allowed values, and cross-field constraints must be satisfied.

The validation error messages are detailed and structured, reporting the field path, expected type, and actual value for each violation. This specificity is important for the LLM retry loop: when extraction fails validation, the error message provides actionable feedback. For example, a snippet validation error reports "Value-snippet mismatches (possible hallucination): input 'cell_count_mean' has value 4.37 but snippet contains '3.21'. Check that extracted values match the source text exactly." This tells the LLM both what failed and how to fix it.

SubmodelTarget uses discriminated unions for its 15 forward model types: the `type` field determines which additional fields are required, ensuring type-specific validation without manual dispatch logic. CalibrationTarget uses a class hierarchy with shared base models for source documentation, relevance assessment, and experimental context.

Supplementary Section 5 provides the complete schema definitions for both schemas.

2.3 Validation Framework

Both schemas share a common validation core (DOI resolution, value-in-snippet matching, unit parsing, source relevance checks) with schema-specific extensions. Each validator runs independently, and a target must pass all validators before proceeding to downstream use.

2.3.1 Reference Validation

LLMs sometimes generate references to entities that do not exist elsewhere in the document. Reference validators check that every `uses_inputs` reference matches a defined input name, every `source_ref` matches a defined source, parameters referenced in the model exist in the parameter list, and observable definitions reference valid state variables.

2.3.2 Content Validation

Content validators address fabrication of values, citations, and other factual claims.

DOI validation queries CrossRef for every DOI in the sources list. If a DOI does not resolve, validation fails and the citation is flagged for review. Title validation further checks that the

extracted paper title fuzzy-matches the CrossRef metadata (using a 75% similarity threshold), catching cases where a valid DOI is paired with fabricated bibliographic details.

Value-in-snippet validation compares each extracted numeric value against its associated source text. If an input claims a value of 4.37 but the quoted snippet contains 3.21, validation fails. This is the primary anti-hallucination defense in the framework. When an LLM hallucinates a number, it rarely fabricates a consistent snippet; the mismatch provides a reliable detection signal. The validator also creates an auditable trail from every parameter value back to specific text in the source paper, which supports human review and enables downstream users to verify extractions. A separate external validator (described in Section 2.3.3) further verifies that snippets actually appear in the cited source papers, catching cases where the LLM fabricates plausible snippets wholesale.

Unit validation parses all unit strings using Pint [27], a Python library for defining, operating on, and converting between physical units (e.g., verifying that "cells/mm^2" and "1/day" are dimensionally valid). Malformed or unrecognized units trigger failure.

Code validation goes beyond syntax checking. The validator parses the code structure, then executes it with mock data to verify correct behavior. For `observation_code`, execution must return the required dictionary keys (`value`, `sd`, `sd_uncertain`) with appropriate types. For observable code, execution must return arrays with correct units and dimensions. This catches logic errors and signature mismatches before inference.

Hardcoded constant detection scans code blocks for numeric literals that should be documented inputs. The parser identifies constants (e.g., `1440` for a minutes-to-days conversion) that are not in the allowed set (small integers 0–5, common fractions, statistical percentiles). Each flagged constant must either be moved to an input with provenance or documented as an `observable_constant` with a `biological_basis` explanation.

Scale consistency validation executes observable code with mock species data spanning a 1000-fold range. If the output is constant despite varying inputs, or if the output magnitude suggests a ratio/score mismatch (e.g., values near 100 when 0–1 is expected), validation warns of potential unit or scaling errors.

Source relevance validators enforce the translation requirements described in Section 2.2.5. Non-peer-reviewed sources are flagged by default (see Section 2.2.5). Cross-species, cross-indication, and low TME compatibility extractions must specify minimum uncertainty thresholds (2-fold, 3-fold,

and 10-fold respectively), ensuring that translation uncertainty propagates to prior widths.

Control character validation scans all string fields for characters that corrupt YAML parsing (ASCII control codes, non-breaking spaces, and similar). PDF copy-paste often introduces invisible characters that cause parsing failures; this validator catches them before they propagate.

Each validator raises a specific exception class from a hierarchy organized by failure mode: `SnippetValueMismatchError` for hallucinated values, `DOIResolutionError` for fabricated citations, `CodeExecutionError` for code that fails at runtime, `UnitParsingError` for invalid units, and so on. Each exception class carries a `category` attribute (hallucination, fabrication, code, units, reference, structural, prior) that enables aggregation across extraction runs. The exception messages are designed for the LLM retry loop, including both the specific failure and guidance for correction. This structured exception hierarchy also feeds into the extraction metrics reported in Section 3. Optional Logfire instrumentation [28] provides OpenTelemetry-based tracing of LLM calls and validation steps for debugging extraction failures.

2.3.3 External Validation

The validators above run automatically during schema parsing. An additional external validator verifies that value snippets actually appear in the cited source papers. This validator fetches paper text (abstract and full text when available) from Europe PMC and Unpaywall using the DOI, then uses fuzzy matching (80% similarity threshold) to locate each snippet in the retrieved text. For NIH-funded research, PubMed Central deposits are accessible through the Europe PMC API, providing full-text coverage for a substantial fraction of biomedical literature; Unpaywall further extends access to open-access versions hosted by publishers. Snippets that cannot be found in the source paper are flagged for review. This catches a subtle failure mode where the LLM fabricates plausible-looking snippets that contain the correct value but do not actually appear in the cited paper. Because this validation requires network access to external APIs, it runs as a separate manual step rather than during schema parsing.

2.3.4 Structural Validation

Structural validators check schema-level constraints: time spans must have start before end, required fields for each model type must be present, and measurement arrays must have consistent

dimensions. Supplementary Section 3 provides implementation details for each validator.

2.3.5 CalibrationTarget-Specific Validation

CalibrationTarget adds validators for full-model observables. Observable code is executed against mock species data to verify correct function signatures, output dimensionality, and unit consistency. A denominator audit requires that density or fraction observables explicitly document what tissue area or cell population serves as the denominator, both experimentally and in the model, catching a common source of order-of-magnitude errors. Species existence checks verify that every species referenced in the observable code exists in the model structure. Distribution code is executed with the declared inputs to verify that computed summary statistics (median and CI95) match the values reported in the empirical data block within tolerance (1% for median, 10% for CI bounds), catching transcription errors and code bugs before they propagate to calibration.

2.4 Code Generation

Validated SubmodelTarget specifications can be automatically translated into Julia scripts for Bayesian inference using Turing.jl [29]. The translator generates self-contained scripts with observed data constants, forward model functions, prior distributions, and likelihood terms. When multiple targets share parameters (identified by matching parameter names), the translator generates a joint model that declares each shared parameter once and constrains it from all relevant data sources simultaneously, yielding tighter posteriors than independent calibration. We chose to generate complete, readable scripts rather than provide a library abstraction, so that users can inspect, modify, and debug the inference model directly. Supplementary Section 4 provides details on the generated code structure, forward model handling, and inference diagnostics.

The two schemas connect through a natural calibration pipeline: SubmodelTarget posteriors from isolated experiments become informative priors for full-model calibration against CalibrationTarget endpoints. Because SubmodelTarget parameter names match the full model, the posterior distributions transfer directly without post-hoc mapping.

2.5 Application: PDAC QSP Model

The PDAC QSP model used in this study is adapted from a spatial QSP model of hepatocellular carcinoma [30] that first incorporated fibroblast dynamics into the tumor microenvironment. We evaluate MAPLE on this model using both schemas: 18 SubmodelTargets for individual parameter calibration from isolated experiments (spanning tumor growth, stromal dynamics, immune recruitment, and cytokine signaling), and 50 CalibrationTargets for full-model endpoints (baseline tumor microenvironment composition, neoadjuvant immunotherapy response, and clinical progression).

2.5.1 Evaluation Metrics

We report metrics across four stages of the workflow:

Extraction performance. First-attempt validation pass rate, retry distribution, average extraction time, and token usage per target.

Error analysis. Distribution of validation failures by category (hallucination, fabrication, units, code, prior specification), and tool usage patterns (web search and code execution calls).

Source characteristics. Distribution of source types (human clinical, human in vitro, animal in vivo, animal in vitro), species translation requirements (human→human vs. cross-species), and indication match (exact, related, or proxy).

Curation analysis. Post-extraction revision rates for both collaboration modes (batch pipeline with subsequent curation vs. interactive extraction), and classification of edits by content type (scientific, narrative, structural, metadata) to characterize the relative contributions of LLM extraction and modeler judgment.

3 Results

3.1 SubmodelTarget Extraction and Validation

We applied MAPLE to extract SubmodelTargets for a PDAC QSP model, covering tumor growth, stromal dynamics, immune recruitment, and cytokine signaling. An initial batch of 18 targets was

extracted through the automated retry loop (Table 1); additional targets were extracted interactively during curation, yielding a final curated set of 37 SubmodelTargets used for joint inference. The metrics below characterize the automated first pass; Section 3.2 discusses the subsequent human curation step.

Table 1: Extraction metrics per calibration target.

Target	Retries	Duration (min)	Tokens	Cost
N_IL2_CD4	3	6.5	198,267	$0.33
N_IL2_CD8	1	3.4	98,233	$0.19
initial_tumour_diameter	1	3.0	70,219	$0.15
k_C1_growth	1	4.1	81,606	$0.19
k_CCL2_sec	5	9.2	367,754	$0.54
k_ECM_apsc_sec	5	10.8	285,309	$0.57
k_ECM_deg	1	5.8	135,317	$0.30
k_ECM_qpsc_sec	5	12.0	406,891	$0.56
k_M1_pol	1	7.1	190,149	$0.40
k_MDSC_rec	3	8.4	221,048	$0.42
k_Mac_rec	3	10.0	239,265	$0.42
k_apsc_death	2	5.4	134,626	$0.26
k_psc_activation	7	9.9	354,840	$0.47
k_psc_const	2	7.7	209,064	$0.44
k_qpsc_death	1	3.8	91,974	$0.18
n_CD8_clones	1	5.3	114,839	$0.27
n_Treg_clones	2	5.7	133,558	$0.28
q_Treg_T_in	6	23.9	586,283	$0.77
Average	**2.8**	**7.9**	**217,735**	**$0.37**

None of the extractions passed validation on the first attempt; all required at least one automated retry, where validation errors were fed back to the LLM without human intervention. Seven targets succeeded after a single retry, while parameters like CCL2 secretion and ECM production required up to 7 automated iterations to resolve unit ambiguities or reconcile conflicting literature. Each extraction took 7.9 minutes on average, consuming 217,735 tokens per target (3.9M total).

The validators caught 24 errors during extraction (Table 2). Unit errors were the most common, reflecting inconsistent unit conventions across experimental papers. Fabrication errors occurred when the LLM generated plausible but invalid DOIs, which the DOI resolution validator rejected. Prior specification errors arose when proposed uncertainty ranges did not match the stated translation uncertainty. Code errors were caught when observation functions failed unit checks.

The LLM used 210 web searches to find and verify sources, and 22 code executions to check

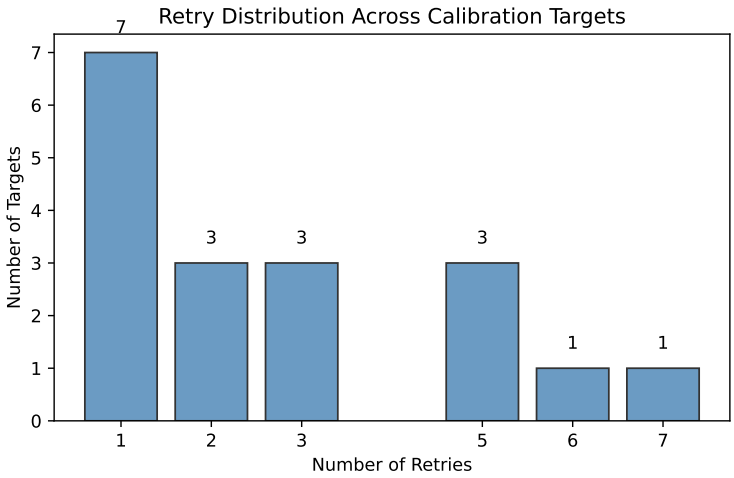

Figure 1: Distribution of automated retry counts across 18 extraction attempts.

Table 2: Error categories encountered during extraction. Each target may have multiple errors.

Error Category	Count	%
units	9	38%
prior	5	21%
fabrication	4	17%
code	4	17%
other	1	4%
hallucination	1	4%
Total	**24**	

observation functions.

3.1.1 Source Characteristics

The 37 curated targets drew from a range of experimental sources (Supplementary Section 8). Human clinical data accounted for 45% of targets, with the remainder from animal studies (in vivo and in vitro). About 56% of targets used human data directly; the rest drew from mouse (35%) or rat (8%) experiments, requiring cross-species scaling before use in a human QSP model. About 45% of targets came from PDAC-specific experiments; most of the remainder (43%) relied on proxy indications such as other cancer types, chronic pancreatitis, or fibrosis models with similar biology.

3.2 Human Curation

The automated retry loop resolves formatting errors, reference mismatches, and unit inconsistencies, but the resulting extractions are starting points rather than finished products. To quantify the extent of human curation, we compared the raw LLM output against the final curated versions. All curation was performed interactively with Claude Code (claude-opus-4-6), with the modeler directing edits, assessing biological plausibility, and making modeling decisions while the LLM handled text generation, code writing, and literature comprehension.

No batch extraction was used without modification. In a documented review of one extraction batch (12 CalibrationTarget files), 58% were rejected outright for reasons including placeholder outputs, mislabeled quantities, and order-of-magnitude disagreements with accepted sources.

Comparing the 37 batch-extracted SubmodelTarget files against the raw LLM output reveals the scope of modeler intervention. The modeler changed the forward model type in 65% of files (24/37), selecting a different modeling approach for the same experimental data. Prior distribution parameters (distribution type, location, scale, or bounds) were adjusted in 46% of files. Input data values were changed in 46% of files, and 32% had inputs added or removed. Source relevance assessments (species translation, indication match, uncertainty quantification) were revised in every file.

For the 22 batch-extracted CalibrationTargets, the modeler revised observable code (species-to-measurement mappings) in 32% of files, changed the observable species list in 23%, and adjusted empirical data values in 18%. Source relevance assessments were changed in 36% of files.

The remaining 27 CalibrationTargets were extracted through a different collaboration mode: interactive extraction with Claude Code using source PDFs loaded directly into the session. In this mode, the modeler and LLM collaborated on extraction and curation simultaneously, with the modeler providing domain guidance (species mapping, denominator definitions, distribution choices) while the LLM handled literature comprehension and code generation. These files required minimal subsequent revision (a single shared reference value correction across 7,483 total lines). The near-zero revision rate does not indicate less human input; rather, modeler judgment was embedded in the extraction process, producing near-final quality directly.

The schema and validators remain valuable throughout both collaboration modes. Schema validation catches errors introduced during manual editing (misspelled species names, broken cross-references, unit mismatches), and the value-in-snippet requirement ensures that edits to numeric values maintain traceability to source text.

3.3 CalibrationTarget Extraction

We applied the CalibrationTarget schema to 50 full-model endpoints for the same PDAC QSP model, organized into three scenario groups: 31 baseline (no treatment) targets capturing the untreated tumor microenvironment, 18 neoadjuvant immunotherapy targets (GVAX ± nivolumab), and 1 clinical progression target (Table 3). All 50 targets pass schema validation including observable code execution, distribution code verification, and unit consistency checks.

Table 3: CalibrationTarget summary by scenario group. Baseline targets capture untreated tumor microenvironment composition (immune cell densities, stromal fractions, cytokine levels). Neoadjuvant immunotherapy targets capture response to GVAX vaccine with or without nivolumab (anti-PD-1). Clinical progression captures tumor growth dynamics.

Scenario Group	Targets
Baseline (no treatment)	31
Neoadjuvant immunotherapy (GVAX ± nivolumab)	18
Clinical progression	1
Total	**50**

Of the 50 targets, 22 were extracted by GPT-5.1 through the batch pipeline and subsequently curated interactively with Claude Code, 27 were extracted interactively with Claude Opus 4 using source PDFs loaded directly into the session, and 1 was curated manually. The 18 neoadjuvant

immunotherapy targets include per-patient data digitized from scatter plots (30% of all CalibrationTargets), with the modeler directing figure interpretation, pooling strategy, and bootstrap uncertainty quantification while the LLM assembled the CalibrationTarget structure from those data points. Curation metrics for both extraction modes are reported in Section 3.2.

The observables span 25 unique model species across immune, myeloid, stromal, and tumor cell populations, as well as selected microenvironment variables. Observable complexity ranged from 1 to 19 model species per target (mean 7.8), with the most complex observables computing immune subset fractions requiring all modeled populations in the denominator. The targets drew from 21 unique primary sources with publication dates spanning 2000–2026.

Supplementary Section 9 provides detailed per-target metrics including observable categories, species counts, and extraction model attribution.

3.4 Code Generation and Inference

All 18 SubmodelTargets were translated to a joint Julia inference script targeting Turing.jl [29]. The translator identified 19 unique parameters and generated corresponding priors and likelihood terms. All parameters converged ($\hat{R} < 1.01$, ESS > 3000), with posterior estimates yielding biologically plausible values (e.g., tumor doubling time of $\sim$130 days, consistent with clinical PDAC observations). Full inference results including convergence diagnostics, posterior distributions, and model type breakdown are reported in Supplementary Section 10.

4 Discussion

MAPLE demonstrates that structured schemas can serve as a productive collaboration interface between LLMs and modelers for QSP model calibration. The curation analysis reveals that modeler judgment contributes roughly half of the final calibration target content, with the LLM providing the other half as a structured scaffold. Rather than automating away the modeler, the framework restructures the collaboration: the schema defines what must be captured, validators catch errors from both LLM and human sources, and the separation of data extraction from modeling decisions clarifies where different types of expertise are needed. A second key finding is that LLM failure modes are often systematic and detectable: hallucinated values rarely match quoted source text,

fabricated DOIs fail resolution, and malformed code fails execution. By targeting these failure modes with specific validators, the framework catches errors automatically and uses structured feedback to guide correction.

4.1 Validation as the Core Mechanism

None of the 18 SubmodelTarget extractions passed validation on the first attempt, and the same automated retry pipeline was used for batch-extracted CalibrationTargets (detailed retry metrics are reported for SubmodelTargets, where the Logfire-instrumented pipeline provided per-target tracing). This is not a limitation but rather the expected behavior of the system. The validators serve as an automated first-pass review, catching errors that would otherwise require human attention. The retry loop, where validation failures are fed back to the LLM with specific error messages, resolves most issues without human intervention. The extractions that required multiple retries (up to 7) typically involved ambiguous unit conventions or conflicting literature values, situations where the LLM needed multiple attempts to find consistent interpretations.

The value-in-snippet requirement proved particularly effective. By requiring that every extracted number appear verbatim in quoted source text, we create an auditable trail and catch a common LLM behavior: paraphrasing. The LLM typically extracts accurate content on the first attempt, but rephrases source text rather than quoting it exactly. The validator flags these cases, and the retry loop produces verbatim snippets. A separate external validator then checks that snippets actually appear in the cited papers, catching cases where the LLM generates plausible text that does not exist in the source. Together, these validators support human review: a reviewer can verify each extraction by checking the snippet against the source paper, without re-reading entire methods sections.

4.2 Schema Design Choices

The separation of data extraction from model specification reflects a deliberate design philosophy. The inputs layer captures what the paper reports, with no modeling decisions; the calibration layer specifies how that data constrains parameters. This separation has several benefits. First, it clarifies where human judgment is required: the inputs should be objective (what numbers appear in the paper), while the calibration layer involves modeling choices (what forward model to use,

what prior width is appropriate). Second, it enables reuse: if a modeling assumption changes, the inputs remain valid and only the calibration layer needs revision. Third, it supports review: a reviewer can verify inputs by checking source text, while a modeler can assess the calibration specification independently.

The dual-schema architecture applies the same data-first principles at two scales of calibration. SubmodelTarget operates on isolated experiments that constrain individual parameters through simplified forward models; CalibrationTarget operates on clinical and in vivo endpoints that constrain the integrated model through full-species observables. Shared design elements (source provenance, relevance assessment, experimental context, value-snippet validation) ensure consistent rigor across both scales, while schema-specific components (forward model types vs. observable code, error models vs. distribution code) capture the distinct requirements of each calibration regime.

The forward model abstraction similarly separates concerns. Built-in templates for common patterns reduce the opportunity for error in routine cases, while custom code blocks provide flexibility for complex situations and remain subject to validation.

4.3 Limitations of Extracted Data

The source characteristics reveal a fundamental challenge in QSP model calibration. Among SubmodelTargets, 45% came from PDAC-specific experiments, while the remainder relied on proxy or related indications. Similarly, 35% required cross-species translation from mouse data. CalibrationTargets face analogous challenges: published PDAC clinical endpoints with the granularity needed for QSP calibration (e.g., per-compartment cell counts, immune subset fractions) are limited, and many baseline tumor microenvironment measurements rely on small cohorts or related indications. These are not failures of the extraction process but rather reflections of available literature: disease-specific experiments measuring the exact quantities needed for model calibration are insufficient to fully constrain all parameters.

The schema addresses this reality through explicit documentation. The `source_relevance` block requires that cross-species and cross-indication extractions specify translation uncertainty, which propagates to prior widths during inference. This does not solve the underlying data gap, but it makes assumptions explicit and quantifies their impact on parameter uncertainty. A parameter calibrated from mouse fibrosis data with 5-fold translation uncertainty will have a correspondingly

wide posterior, appropriately reflecting the limited applicability of available evidence.

4.4 Comparison to Manual Curation

Manual curation by modelers remains the gold standard for parameter extraction quality. An experienced modeler brings biological intuition, can assess methodological quality, and can make nuanced judgments about applicability. The curation results in Section 3.2 quantify this contribution across both schemas. For SubmodelTargets, the modeler changed forward model types, adjusted prior distributions, revised input data, and rewrote source relevance assessments across the majority of batch-extracted files. For CalibrationTargets, the modeler revised observable code (species-to-measurement mappings), corrected species lists, and adjusted empirical data values. In both cases, these are scientific decisions that shaped the final calibration targets, not formatting corrections. MAPLE does not reduce the need for this expertise; rather, it restructures how modeler and LLM collaborate through a shared structured interface.

In manual curation, the modeler performs all tasks: literature search, data extraction, quality assessment, unit conversion, uncertainty estimation, and documentation. This conflates tasks with different skill requirements and error profiles. Literature search benefits from domain knowledge; numeric extraction is tedious and error-prone; documentation is often neglected under time pressure. The structured schema addresses this last problem directly: every input requires a source reference and snippet, every prior requires a rationale, every cross-indication extraction requires a justification. This creates a record that survives personnel changes and enables future review.

The two collaboration modes observed in this study offer different trade-offs. In the batch pipeline mode, the LLM produces a structured draft that the modeler then curates interactively: selecting appropriate forward model types and prior distributions for SubmodelTargets, correcting observable code and species mappings for CalibrationTargets, and revising source relevance assessments for both. This mode is efficient for initial coverage (many files per run) but carries a high rejection rate (58% in a documented batch review) because the LLM lacks real-time modeler guidance on source selection, species mapping, and denominator definitions. In the interactive mode, the modeler and LLM work together from the start, with the modeler directing extraction decisions while the LLM handles text generation, code writing, and literature comprehension. This mode produces near-final quality but requires deeper per-file engagement.

In both modes, the schema serves as the collaboration interface. It defines what information must be captured (ensuring completeness), constrains how it is represented (enabling validation), and separates data extraction from modeling decisions (clarifying where different types of expertise are needed). The validators provide guardrails during both LLM extraction and human editing, catching errors regardless of their source.

4.5 Relationship to Existing Approaches

MAPLE addresses a specific gap in the landscape of LLM-assisted extraction tools. General-purpose biomedical extraction frameworks like LLM-IE [20] provide building blocks for named entity recognition and relation extraction, but do not address the downstream requirements of QSP calibration: uncertainty quantification, forward model specification, and code generation for inference. Interactive systems like SciDaSynth [21] support structured data extraction with human-in-the-loop refinement, but focus on tabular output rather than the hierarchical schema needed for QSP model calibration. QSP-Copilot [19] applies LLMs to QSP model development broadly, including literature extraction, but emphasizes model structure discovery (entity-interaction pairs) rather than quantitative parameter calibration with provenance.

The validation approach in MAPLE draws on established strategies for hallucination detection. Requiring extracted values to appear in quoted source text is analogous to the grounding strategies used in retrieval-augmented generation [25], where LLM outputs are verified against retrieved documents. The schema-based validation enforces structural constraints similar to those in constrained decoding and structured output systems, where JSON schemas guide LLM generation toward valid output [31]. The key contribution is combining these strategies with domain-specific validators (DOI resolution, unit parsing, code execution) in a framework tailored to quantitative scientific extraction.

Recent work on knowledge graph construction from biomedical literature [16, 13] demonstrates that LLMs can extract structured relationships at scale. The Immune Cell Knowledge Graph (ICKG) used zero-shot prompting with Llama 3.1 to extract activation and inhibition relationships from over 24,000 PubMed abstracts. Such approaches prioritize coverage and can tolerate some extraction errors because downstream analyses aggregate across many relationships. Parameter calibration has different requirements: each extracted value directly influences model predictions,

so individual errors matter more. MAPLE trades some throughput for higher per-target validation stringency.

The IQ Consortium has outlined best practices for AI/ML in quantitative modeling, including the use of NLP for knowledge extraction from biomedical literature [7]. They emphasize explainability, human-in-the-loop oversight, and consideration of context of use. MAPLE aligns with these principles: the schema documents reasoning and provenance, validators catch errors before human review, and the generated code is transparent and modifiable. The growing interest in coupling QSP with machine learning [6, 32, 8, 9] suggests that hybrid approaches combining mechanistic modeling with data-driven methods will become increasingly important; automated literature extraction is one component of this broader integration. Recent work has also explored using LLMs directly as tools for comparing and synthesizing perspectives on QSP methodology [10]. While we evaluate MAPLE on a QSP model, the approach is not limited to pharmacology: any parameter-rich mechanistic model that draws calibration data from literature (e.g., systems biology signaling networks, physiologically-based models) could benefit from structured extraction with validation.

4.6 Limitations and Future Work

Several limitations merit discussion. First, the framework currently relies on the LLM's built-in web search to find and access source literature. This means extraction quality is bounded by the LLM's ability to locate relevant papers and read their content. Closed-access publications may be inaccessible or only partially available (e.g., abstract only), limiting both source discovery and the depth of extraction. Future work should integrate retrieval-augmented approaches that provide the LLM with full-text access to institutional library holdings, enabling extraction from deeper within papers rather than relying on what the LLM can retrieve through web search alone. More broadly, the framework depends on LLM capabilities that continue to evolve: extraction accuracy depends on model capabilities, cost and latency affect practical usability, and extraction is stochastic, with the same query potentially yielding different papers and extractions across runs. The modular architecture allows swapping LLM providers, model versions, and reasoning effort settings, but performance characteristics may differ.

Second, the curation data show that modeler intervention is pervasive, not confined to edge cases. For SubmodelTargets, the modeler changed the forward model type in 65% of files and

adjusted prior parameters in 46%, reflecting scientific judgments about how to model the experiment and what uncertainty is appropriate. Prompt improvements could reduce some curation: better model type selection heuristics, canonical distribution patterns, and more explicit guidance on denominator construction would address recurring error categories. But the core scientific decisions (is this source appropriate? what prior width reflects my uncertainty? how do these species map to the measured quantity?) will remain modeler responsibilities regardless of LLM capability. The schema and validators can also be used without LLMs at all, providing structure and completeness checking for purely manual curation.

Third, CalibrationTargets required different kinds of curation than SubmodelTargets. Observable code operating on full model species (changed in 32% of batch-extracted files), Monte Carlo distribution derivation, and denominator audit fields demand modeling expertise that current LLMs provide as a starting point rather than a finished product. The interactive extraction mode partially addresses this by embedding modeler guidance in real time, but at the cost of deeper per-file engagement. Additionally, extracting structured quantitative data from figures remains challenging; the neoadjuvant immunotherapy targets in this study required manual digitization of scatter plots before the LLM could assemble the CalibrationTarget structure. Multimodal LLMs with vision capabilities are improving rapidly at figure interpretation, and integrating these capabilities into the extraction pipeline is a natural direction for future work.

Finally, the evaluation presented here covers a single model (PDAC QSP) and a limited set of parameters. Broader evaluation across different therapeutic areas and model structures would better characterize where the framework succeeds and where it struggles.

Study Highlights

What is the current knowledge on the topic?

QSP model calibration requires parameter values from experimental literature, but manual curation is labor-intensive with inconsistent documentation practices, often lacking systematic uncertainty quantification and provenance, while LLM extraction exhibits hallucination and fabrication errors that are unacceptable for quantitative modeling.

What question did this study address?

Can structured schemas serve as a productive collaboration interface between LLMs and modelers for QSP model calibration? What is the relative contribution of automated extraction versus modeler judgment in the final calibration targets?

What does this study add to our knowledge?

Structured validation schemas enable two modes of LLM-modeler collaboration for parameter extraction: batch extraction followed by interactive curation, and real-time interactive extraction. Both modes required substantial modeler input: no batch extraction was used without modification, with the modeler changing forward model types in 65% of SubmodelTargets, adjusting prior distribution parameters in 46%, and revising observable code in 32% of CalibrationTargets. Targeted validators (value-in-snippet matching, DOI resolution, code execution) catch characteristic LLM errors in both modes, and automatic code generation enables joint Bayesian inference across calibration targets.

How might this change drug discovery, development, and/or therapeutics?

By structuring the collaboration between LLMs and modelers through validated schemas, the framework produces reproducible, provenance-rich calibration targets regardless of workflow mode. This may improve the consistency and documentation quality of QSP model calibration across modeling teams, while making the modeler contribution explicit and auditable.

Acknowledgments

Funding

The research was funded by the Lustgarten Foundation: Pancreatic Cancer Research.

Conflict of Interest

A.S.P. receives research funding from Merck and Pfizer in the area of QSP modeling; he is also a founder and holds equity in OptaNova Pharma and Terebra Therapeutics outside the work described therein. The terms of these arrangements are being managed by the Johns Hopkins University in accordance with its conflict-of-interest policies. J.E. declares that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential

conflict of interest.

Data Availability Statement

MAPLE (the SubmodelTarget and CalibrationTarget schemas, validation framework, and Julia code generator) is available at `https://github.com/popellab/qsp-llm-workflows`. The PDAC calibration targets used in this study (18 SubmodelTargets and 50 CalibrationTargets) are provided as supplementary YAML files. Generated Julia inference scripts are included in the supplementary materials.

References

[1] K Gadkar, DC Kirouac, DE Mager, PH van der Graaf, and S Ramanujan. A Six-Stage Workflow for Robust Application of Systems Pharmacology. *CPT: Pharmacometrics & Systems Pharmacology*, 5(5):235–249, May 2016. ISSN 2163-8306. doi: 10.1002/psp4.12071.

[2] Hanwen Wang, Theinmozhi Arulraj, Alberto Ippolito, and Aleksander S. Popel. Quantitative Systems Pharmacology Modeling in Immuno-Oncology: Hypothesis Testing, Dose Optimization, and Efficacy Prediction. *Handbook of Experimental Pharmacology*, 289:261–284, 2025. ISSN 0171-2004. doi: 10.1007/164_2024_735.

[3] Hanwen Wang, Theinmozhi Arulraj, Alberto Ippolito, and Aleksander S. Popel. From virtual patients to digital twins in immuno-oncology: Lessons learned from mechanistic quantitative systems pharmacology modeling. *npj Digital Medicine*, 7(1):189, July 2024. ISSN 2398-6352. doi: 10.1038/s41746-024-01188-4.

[4] Theodore R. Rieger, Richard J. Allen, Lukas Bystricky, Yuzhou Chen, Glen Wright Colopy, Yifan Cui, Angelica Gonzalez, Yifei Liu, R. D. White, R. A. Everett, H. T. Banks, and Cynthia J. Musante. Improving the generation and selection of virtual populations in quantitative systems pharmacology models. *Progress in Biophysics and Molecular Biology*, 139:15–22, November 2018. ISSN 0079-6107. doi: 10.1016/j.pbiomolbio.2018.06.002.

[5] Morgan Craig, Jana L. Gevertz, Irina Kareva, and Kathleen P. Wilkie. A practical guide for

the generation of model-based virtual clinical trials. *Frontiers in Systems Biology*, 3, June 2023. ISSN 2674-0702. doi: 10.3389/fsysb.2023.1174647.

[6] Núria Folguera-Blasco, Florencia A. T. Boshier, Aydar Uatay, Cesar Pichardo-Almarza, Massimo Lai, Jacopo Biasetti, Richard Dearden, Megan Gibbs, and Holly Kimko. Coupling quantitative systems pharmacology modelling to machine learning and artificial intelligence for drug development: Its pAIns and gAIns. *Frontiers in Systems Biology*, 4, July 2024. ISSN 2674-0702. doi: 10.3389/fsysb.2024.1380685.

[7] Nadia Terranova, Didier Renard, Mohamed H. Shahin, Sujatha Menon, Youfang Cao, Cornelis E.C.A. Hop, Sean Hayes, Kumpal Madrasi, Sven Stodtmann, Thomas Tensfeldt, Pavan Vaddady, Nicholas Ellinwood, and James Lu. Artificial Intelligence for Quantitative Modeling in Drug Discovery and Development: An Innovation and Quality Consortium Perspective on Use Cases and Best Practices. *Clinical Pharmacology & Therapeutics*, 115(4):658–672, 2024. ISSN 1532-6535. doi: 10.1002/cpt.3053.

[8] Igor Goryanin, Irina Goryanin, and Oleg Demin. Revolutionizing drug discovery: Integrating artificial intelligence with quantitative systems pharmacology. *Drug Discovery Today*, 30(9): 104448, September 2025. ISSN 1878-5832. doi: 10.1016/j.drudis.2025.104448.

[9] Ioannis P. Androulakis, Lourdes Cucurull-Sanchez, Anna Kondic, Krina Mehta, Cesar Pichardo, Meghan Pryor, and Marissa Renardy. The dawn of a new era: Can machine learning and large language models reshape QSP modeling? *Journal of Pharmacokinetics and Pharmacodynamics*, 52(4):36, June 2025. ISSN 1573-8744. doi: 10.1007/s10928-025-09984-5.

[10] Ioannis P. Androulakis, Limei Cheng, Carolyn R. Cho, and Tongli Zhang. Leveraging large language models to compare perspectives on integrating QSP and AI/ML. *Journal of Pharmacokinetics and Pharmacodynamics*, 52(3):29, May 2025. ISSN 1573-8744. doi: 10.1007/s10928-025-09976-5.

[11] Sendong Zhao, Chang Su, Zhiyong Lu, and Fei Wang. Recent advances in biomedical literature mining. *Briefings in Bioinformatics*, 22(3):bbaa057, May 2021. ISSN 1477-4054. doi: 10.1093/bib/bbaa057.

[12] Jiacheng Hu, Runyuan Bao, Yang Lin, Hanchao Zhang, and Yanlin Xiang. Accurate Medical Named Entity Recognition Through Specialized NLP Models, December 2024.

[13] Ting Gao, Xue Zhai, Chuan Yang, Linlin Lv, and Han Wang. Joint extraction of entity and relation based on fine-tuning BERT for long biomedical literatures. *Bioinformatics Advances*, 4(1):vbae194, January 2024. ISSN 2635-0041. doi: 10.1093/bioadv/vbae194.

[14] Jingcheng Du, Dong Wang, Bin Lin, Long He, Liang-Chin Huang, Jingqi Wang, Frank J. Manion, Yeran Li, Nicole Cossrow, and Lixia Yao. Use of deep learning-based NLP models for full-text data elements extraction for systematic literature review tasks. *Scientific Reports*, 15 (1):19379, June 2025. ISSN 2045-2322. doi: 10.1038/s41598-025-03979-5.

[15] Ferran Gonzalez Hernandez, Quang Nguyen, Victoria C. Smith, José Antonio Cordero, Maria Rosa Ballester, Màrius Duran, Albert Solé, Palang Chotsiri, Thanaporn Wattanakul, Gill Mundin, Watjana Lilaonitkul, Joseph F. Standing, and Frank Kloprogge. Named entity recognition of pharmacokinetic parameters in the scientific literature. *Scientific Reports*, 14 (1):23485, October 2024. ISSN 2045-2322. doi: 10.1038/s41598-024-73338-3.

[16] Shan He, Yukun Tan, Qing Ye, Matthew M Gubin, Merve Dede, Hind Rafei, Weiyi Peng, Katayoun Rezvani, Vakul Mohanty, and Ken Chen. AI-powered Immune Cell Knowledge Graph (ICKG) with granular immune contexts enables immune program interpretation. *npj Artificial Intelligence*, 2(1):13, January 2026. ISSN 3005-1460. doi: 10.1038/s44387-025-00060-4.

[17] Muhammad Ali Khan, Umair Ayub, Syed Arsalan Ahmed Naqvi, Kaneez Zahra Rubab Khakwani, Zaryab Bin Riaz Sipra, Ammad Raina, Sihan Zou, Huan He, Seyyed Amir Hossein, Bashar Hasan, R. Bryan Rumble, Danielle S. Bitterman, Jeremy L. Warner, Jia Zou, Amye J. Tevaarwerk, Konstantinos Leventakos, Kenneth L. Kehl, Jeanne M. Palmer, M. Hassan Murad, Chitta Baral, and Irbaz Bin Riaz. Collaborative Large Language Models for Automated Data Extraction in Living Systematic Reviews. *medRxiv: The Preprint Server for Health Sciences*, page 2024.09.20.24314108, September 2024. doi: 10.1101/2024.09.20.24314108.

[18] Aline Gendrin-Brokmann, Eden Harrison, Julianne Noveras, Leonidas Souliotis, Harris Vince, Ines Smit, Francisco Costa, David Milward, Sashka Dimitrievska, Paul Metcalfe, and Emilie

Louvet. Investigating deep-learning NLP for automating the extraction of oncology efficacy endpoints from scientific literature. *Intelligence-Based Medicine*, 10:100152, January 2024. ISSN 2666-5212. doi: 10.1016/j.ibmed.2024.100152.

[19] Anuraag Saini and Ali Farnoud. QSP-Copilot: An AI-Augmented Platform for Accelerating Quantitative Systems Pharmacology Model Development. *CPT: Pharmacometrics & Systems Pharmacology*, 14(11):1775–1786, 2025. ISSN 2163-8306. doi: 10.1002/psp4.70127.

[20] Enshuo Hsu and Kirk Roberts. LLM-IE: A python package for biomedical generative information extraction with large language models. *JAMIA Open*, 8(2):ooaf012, April 2025. ISSN 2574-2531. doi: 10.1093/jamiaopen/ooaf012.

[21] Xingbo Wang, Samantha L. Huey, Rui Sheng, Saurabh Mehta, and Fei Wang. SciDaSynth: Interactive Structured Data Extraction From Scientific Literature With Large Language Model. *Campbell Systematic Reviews*, 21(4):e70073, 2025. ISSN 1891-1803. doi: 10.1002/cl2.70073.

[22] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, June 2024. ISSN 1476-4687. doi: 10.1038/s41586-024-07421-0.

[23] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2):1–55, March 2025. ISSN 1046-8188, 1558-2868. doi: 10.1145/3703155.

[24] Shrey Pandit, Jiawei Xu, Junyuan Hong, Zhangyang Wang, Tianlong Chen, Kaidi Xu, and Ying Ding. MedHallu: A Comprehensive Benchmark for Detecting Medical Hallucinations in Large Language Models, February 2025.

[25] Yihan Li, Xiyuan Fu, Ghanshyam Verma, Paul Buitelaar, and Mingming Liu. Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems, October 2025.

[26] Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, Serge Matveenko, Marcelo Trylesinski, Sydney Runkle, David Hewitt, Alex Hall, and Victorien Plot. Pydantic Validation, October 2025. URL `https://github.com/pydantic/pydantic`.

[27] Hernán E. Grecco. Pint: Operate and manipulate physical quantities in Python, 2025. URL `https://github.com/hgrecco/pint`.

[28] Pydantic. Logfire: Observability platform for Python applications, 2025. URL `https://pydantic.dev/logfire`.

[29] Hong Ge, Kai Xu, and Zoubin Ghahramani. Turing: A Language for Flexible Probabilistic Inference. In *Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics*, pages 1682–1690. PMLR, March 2018.

[30] Shuming Zhang, Hanwen Wang, Yeonju Cho, Wendy Wong, Mark Yarchoan, Elizabeth M. Jaffee, Won Jin Ho, Luciane T. Kagohara, Elana J. Fertig, Aleksander S. Popel, and Atul Deshpande. Quantitative Calibration of a Spatial QSP Model Identifies Fibroblast Impact on HCC Immunotherapy, November 2025. ISSN 2692-8205.

[31] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models, February 2025.

[32] Tomasz Pawłowski, Grzegorz Bokota, Georgia Lazarou, Andrzej M. Kierzek, and Jacek Sroka. Emulation of Quantitative Systems Pharmacology models to accelerate virtual population inference in immuno-oncology. *Methods (San Diego, Calif.)*, 223:118–126, March 2024. ISSN 1095-9130. doi: 10.1016/j.ymeth.2023.12.006.